RemaNet LCA Tool

User guide, example results, and API setup

Repository: github.com/Norani99/Remanet_LCA

November 2026

1. What this tool does

The RemaNet LCA tool matches a short text description of a remanufacturing step, such as “clean and regrind a worn HSS twist drill”, to a background life-cycle sector and returns a GWP (global warming potential) figure in kg CO2-eq. It does this without the contributing company sharing its raw process data.

Matching runs through four steps, cheapest first:

1. **Cache** — an exact repeat of a description seen before.
2. **Knowledge base** — a curated list of known process phrases.
3. **Semantic search** — finds the closest known phrase using sentence embeddings.
4. **Language model** — only used when the first three steps find nothing good.

If none of the four steps gives a confident answer, the tool does not guess. It flags the step as **NEEDS_INPUT** so a person can resolve it by hand. This is the main design choice behind the tool: a step that is wrongly matched with high confidence is worse than a step that is honestly flagged as unresolved.

The version in the public repository ships with made-up (synthetic) example data only. No real company data is included.

2. Installing and running it

2.1 Install

```
git clone https://github.com/Norani99/Remanet_LCA.git
cd Remanet_LCA
pip install -r requirements.txt
```

2.2 Run the example batch

```
python -m remanet_lca.pipeline \
--input data/synthetic/example_processes.csv \
--output results.csv
```

This reads a CSV of process descriptions and writes back one row per step, with the matched sector code, which of the four steps resolved it, a confidence score, and the GWP figure per unit.

2.3 Run it from Python on a single description

```
from remanet_lca.pipeline import MatchingPipeline

pipeline = MatchingPipeline()
result = pipeline.match("clean and regrind a worn HSS twist drill")
print(result)
```

3. What the example data gives you

The seven rows in `data/synthetic/example_processes.csv` cover three products: an HSS twist drill, an electric motor, and a carbide cutting insert. Running the tool on them with the demo knowledge

base gives:

Process step	Resolved by	Sector	GWP (kg CO2-eq/unit)
Clean and regrind a worn HSS drill	knowledge base	BAFU_METAL_GRIND	0.62
Apply PVD coating after regrinding	needs input	—	—
Disassemble/inspect a motor rotor	knowledge base	EXIO_C27_1	4.10
Rewind copper windings on a stator	knowledge base	EXIO_C27_1	4.10
Recondition a carbide insert (regrind)	knowledge base	BAFU_METAL_GRIND	0.62
Recycle scrap tungsten carbide	needs input	—	—
Unclear legacy process note	needs input	—	—

Four of the seven steps resolve from the small demo knowledge base alone, with no language model call at all. The other three are flagged for a human to resolve, because the demo knowledge base is intentionally small. Adding more entries to `data/synthetic/knowledge_base.yaml`, or turning on a language model (section 4), resolves more of them automatically.

4. Results from the two evaluation case studies

These figures come from the fuller evaluation behind the RemaNet LCA paper (submitted to CIRP LCE 2027), run on the original company data. That data is not in this public repository; the numbers below are shown here as context for what the matching approach achieves at full scale, not as something you can reproduce by running the synthetic demo in section 3.

Two remanufacturing cases were evaluated: refurbishing an electric motor, and regrinding an HSS twist drill. Nine language models were compared — both cloud APIs and models running locally — across 39 runs in total.

Case	Agreement rate	Reference GWP
Electric motor refurbishment	~94.8%	12.67 kg CO2-eq per motor
HSS twist-drill regrinding	~80.5%	—

Three things stood out:

- The ~14 percentage point gap between the two cases traces back to the type of consumables each process uses, not to how complex the process itself is.
- A model can have a high agreement rate and still be wrong in a way that matters: it can produce a large, silent error in the GWP figure instead of flagging the step as uncertain. The model with the lowest agreement rate in this evaluation (Mistral 7B) made its mistakes openly, by escalating to NEEDS_INPUT, rather than quietly.
- Small models run locally (around 7 billion parameters) matched the accuracy of the larger cloud models. That matters for companies that cannot send process data to an external API at all.

In short: how often a model agrees with the reference answer matters less than whether its disagreements are visible. A transparent failure can be caught; a silent one cannot.

5. Connecting a real language model

By default the language-model step is switched off, so the tool runs with no API key and no local model installed — anything the knowledge base and semantic search cannot resolve is simply flagged as NEEDS_INPUT. To let the tool resolve more steps automatically, connect a real backend in `config.yaml`.

5.1 Option A — a local model via Ollama

This keeps all process data on your own machine. No data leaves the company network.

1. Install Ollama and pull a model, e.g. `ollama pull qwen2.5:7b`.
2. Make sure Ollama is running (it listens on port 11434 by default).
3. Edit `config.yaml` as shown below.

```
llm:
enabled: true
backend: "ollama"
model: "qwen2.5:7b"
base_url: "http://localhost:11434"
```

5.2 Option B — a cloud API (OpenAI-compatible)

This covers OpenAI, Mistral, and most other providers that use the same request format. Your process descriptions are sent to that provider, so this is not confidentiality-preserving in the way Option A is — only use it with data you are allowed to share externally.

1. Get an API key from your provider.
2. Set it as an environment variable, e.g. `export LLM_API_KEY=sk-...` (never put the key directly in config.yaml or commit it to the repository).
3. Edit `config.yaml` as shown below.

```
llm:
enabled: true
backend: "openai_compatible"
model: "gpt-4o-mini"
base_url: "https://api.openai.com/v1"
api_key_env: "LLM_API_KEY"
```

For Mistral's API, use `base_url: "https://api.mistral.ai/v1"` and the matching model name instead.

5.3 Confidence threshold

A language-model match is only accepted above the confidence threshold set in `config.yaml` (0.70 by default). Below that, the step is still flagged as NEEDS_INPUT rather than accepted on a weak guess:

```
matching:
llm_confidence_threshold: 0.70
```

5.4 Checking it worked

Run the example batch again (section 2.2). Rows that previously showed `needs_input` in the `rung` column should now show `llm`, with a confidence score and a GWP figure filled in.